YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

LOW LEVEL DESIGN

ON

CLOUD COST INTELLIGENCE FOR HEALTHCARE

SUBMITTED BY

MOHAMMED MUNEEB

REG NO :

23BCAICD063

LH-06

GUIDED BY

MISS SINDHU

SUBMISSION DATE

17-04-2026

Table of Contents

1 Introduction
- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions

2 System Design
- 2.1 Application Design
- 2.2 Process Flow
- 2.3 Information Flow
- 2.4 Components Design
  - 2.4.1 Data Loader Module
  - 2.4.2 Data Processing Module
  - 2.4.3 Anomaly Detection Module
  - 2.4.4 Visualization Module
  - 2.4.5 Insight Generation Module
  - 2.4.6 Recommendation Module
- 2.5 Key Design Considerations
- 2.6 API Catalogue

3 Data Design.
- 3.1 Data Model
- 3.2 Data Access Mechanism
- 3.3 Data Retention Policy
- 3.4 Data Migration

4 Interfaces

5 State and Session Management

6 Caching

7 Non-Functional Requirements
- 7.1 Security Aspects
- 7.2 Performance Aspects
- 7.3 Scalability Aspects

8 References

1. Introduction

1.1 Purpose

This document explains the detailed implementation of the Cloud Cost Intelligence System. It describes how different modules are designed, how data is processed, and how the system generates insights from healthcare cloud cost data.

1.2 Scope

This document includes:

- Module-level design
- Data processing logic
- Visualization workflow
- Insight and recommendation generation
- Internal working of the system

1.3 Definitions

- **Cloud Cost:** Expense incurred for using cloud resources such as compute, storage, and data transfer
- **Anomaly:** A data point that significantly deviates from normal cost patterns
- **Seasonal Effect:** Variation in cloud usage due to healthcare demand (e.g., flu season)
- **Dashboard:** Interactive interface displaying charts and insights

2. System Design

2.1 Application Design

The application follows a layered structure:

- Data Layer: Stores dataset in CSV format
- Processing Layer: Handles data cleaning and analysis using Pandas and NumPy
- Presentation Layer: Built using Streamlit for interactive visualization

Each layer works together to ensure smooth data processing and user interaction.

2.2 Process Flow

1. User uploads dataset (CSV file)
2. System validates required columns
3. Data is cleaned and processed
4. Aggregation is performed based on month and department
5. Anomaly detection logic is applied
6. Insights and recommendations are generated
7. Results are displayed in dashboard

2.3 Information Flow

User → File Upload → Data Processing → Analysis → Visualization → Insights → User

This flow ensures that raw data is converted into meaningful information step by step.

2.4 Component Design

2.4.1 Data Loader Module

- Reads CSV file using Pandas
- Converts data into DataFrame

Code Snippet**:**

```
def load_data(file):
    return pd.read_csv(file)
```

2.4.2 Data Processing Module

- Cleans missing values
- Groups and aggregates data
- Prepares dataset for analysis

Code Snippet:

```
def process_data(df):
    return df.groupby(['month', 'department'])['cost'].sum()
```

2.4.3 Anomaly Detection Module

- Identifies abnormal cost spikes
- Uses statistical threshold method

Logic:

threshold = mean + 2 * std

Steps:

- Calculate mean and standard deviation
- Compare values with threshold
- Mark anomalies

2.4.4 Visualization Module

- Generates charts for better understanding

Charts used:

- Pie Chart: Service-wise cost
- Bar Chart: Department-wise cost

- Line Chart: Monthly trends
- Scatter Plot: Anomalies

2.4.5 Insight Generation Module

- Automatically identifies key patterns

Examples:

- Highest cost department
- Peak usage month
- Seasonal trends
- Anomaly count

2.4.6 Recommendation Module

- Suggests cost optimization strategies

Examples:

- Reduce high-cost services
- Investigate anomalies
- Plan for seasonal demand

2.5 Key Design Considerations

- Simplicity: Easy to understand and use
- Scalability: Can handle larger datasets in future
- Clarity: Visual outputs should be clear
- Relevance: Matches healthcare scenarios

2.6 API Catalogue

No external APIs are used. All operations are performed locally using Python libraries.

3. Data Design

3.1 Data Model

Dataset fields include:

- month
- department
- service_type
- cost
- compute_cost
- storage_cost

- transfer_cost
- api_cost
- seasonal_effect

3.2 Data Access Mechanism

- Data is loaded using Pandas
- Processed dynamically in memory

3.3 Data Retention Policy

- Data is temporary
- Cleared after session ends

3.4 Data Migration

- Not required (standalone dataset)

4. Interfaces

- Web-based interface using Streamlit
- Allows file upload and visualization interaction

5. State and Session Management

State and session management is handled using Streamlit's session state mechanism.

- The uploaded dataset is stored temporarily during the session
- Processed data is reused across different visualizations without reloading
- User interactions such as filters and selections are maintained throughout the session
- Session resets when the application is refreshed or closed

6. Caching

Caching is implemented using Streamlit's caching mechanism to improve performance.

- Frequently used functions like data loading and processing can be cached
- Prevents repeated computation when the same dataset is used
- Reduces execution time for large datasets
- Ensures faster dashboard responsiveness

7. Non-Functional Requirements

7.1 Security Aspects

- The system does not process sensitive patient data, ensuring minimal risk
- Input validation is performed during file upload to prevent incorrect or corrupted data

- Data is processed locally within the application without external transmission
- No third-party integrations are used, reducing exposure to external threats

7.2 Performance Aspects

- Data processing is optimized using efficient Pandas operations such as grouping and aggregation
- Visualization rendering is handled by Streamlit for quick and responsive UI updates
- The system performs efficiently for moderate-sized datasets
- Since processing is done in-memory, performance may reduce for very large datasets

7.3 Scalability Aspects

- The system is designed with a modular architecture, allowing easy addition of new features
- Can be extended to handle larger datasets with optimized data processing techniques
- Future integration with cloud storage and distributed processing can improve scalability
- Components such as anomaly detection and visualization can be enhanced independently

8. References

1. Pandas Documentation – https://pandas.pydata.org/
2. NumPy Documentation – https://numpy.org/
3. Streamlit Documentation – https://docs.streamlit.io/
4. Scikit-learn Documentation – https://scikit-learn.org/
5. AWS Cloud Concepts – https://aws.amazon.com/what-is-cloud-computing/